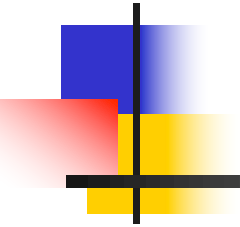
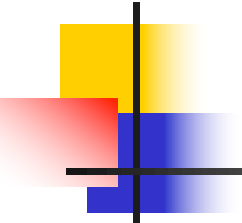


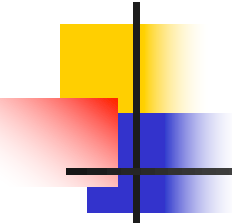
Chap02: C++ Standard IO





todos

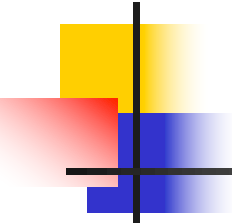
1. C++ Program basic components
2. Understanding `#include <iostream>`
3. Understanding namespace와 `std::`
4. Standard out
5. Standard in



Basic C++ Program

```
/*  
Project Simple  
Print out on the display using cout과 << operator  
*/  
#include <iostream>  
using namespace std;  
int main() {  
    cout << "Hello" << endl; // 화면에 Hello를 출력하고 다음 줄로 넘어감  
    cout << "World.";   
    return 0; // main() 함수가 종료하면 프로그램이 종료됨  
}
```

```
Hello  
World.
```



Comments and main() function

- comments

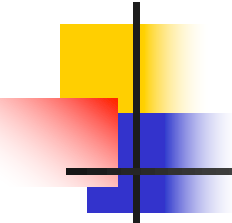
- Memo done by developer
- 프로그램의 실행에 영향을 미치지 않음
 - 여러 줄 주석문 - `/* ... */`
 - 한 줄 주석문 - `//`를 만나면 이 줄의 끝까지 주석으로 처리

- `main()` 함수

- C++ 프로그램의 실행을 시작하는 함수
 - `main()` 함수가 종료하면 C++ 프로그램 종료

- ```
int main() { // main()의 리턴 타입 int

 return 0;
}
```



# #include <iostream>

- #include <iostream>
  - 전처리기(C++ Preprocessor)에게 내리는 지시
    - <iostream> 헤더 파일을 컴파일 전에 소스에 확장하도록 지시
- <iostream> 헤더 파일
  - 표준 입출력을 위한 클래스와 객체, 변수 등이 선언됨
    - ios, istream, ostream, iostream 클래스 선언
    - cout, cin 객체
    - <<, >> 등 연산자 선언

```
#include <iostream>
```

```
....
```

```
std::cout << "Hello" << endl;
```

```
std::cout << "World";
```

# 화면 출력

## ■ cout과 << 연산자 이용

```
std::cout << "Hello" << endl; // 화면에 Hello를 출력하고 다음 줄로 넘어감
```

## ■ cout 객체

- 스크린 출력 장치에 연결된 표준 C++ 출력 스트림 객체
- <iostream> 헤더 파일에 선언
- std 이름 공간에 선언: std::cout으로 사용

## ■ << 연산자

- 스트림 삽입 연산자(stream insertion operator) (피연산자) << (스트림 객체)
  - C++ 기본 산술 시프트 연산자(<<)가 스트림 삽입 연산자로 재정의됨
  - ostream 클래스에 구현됨
  - 오른쪽 피연산자를 왼쪽 스트림 객체에 삽입
  - cout 객체에 연결된 화면에 출력
- 여러 개의 << 연산자로 여러 값 출력

>> : C에서는 비트 연산자로 사용  
C++에서는 삽입연산자  
즉, operator overloading 기능을  
사용하여 <iostream>에 재정의된  
“연산자”

```
std::cout << "Hello\n" << "World.";
```

# << 연산자 활용

상수 ex:

const PI = 3.1415;

- 문자열 및 기본 타입의 데이터 출력
  - `bool`, `char`, `short`, `int`, `long`, `float`, `double` 타입 값 출력

```
int n=3;
char c='#';
std::cout << c << 5.5 << '-' << n << "hello" << true;
```

literal : 자료형을 갖는 상수값

#5.5-3hello1

- 연산식뿐 아니라 함수 호출도 가능

```
std::cout << "n + 5 =" << n + 5;
std::cout << f(); // 함수 f()의 리턴값을 출력한다.
```

- 다음 줄로 넘어가기
  - `endl` 조작자 사용

```
std::cout << "Hello" << std::endl;
```

개행

# cout과 <<를 이용한 화면 출력

```
#include <iostream>

double area(int r); // 함수의 원형 선언

int main() {
 int n=3;
 char c='#';
 std::cout << c << 5.5 << '-' << n << "hello" << true << std::endl;
 std::cout << "n + 5 = " << n + 5 << '\n';
 std::cout << "면적은 " << area(n); // 함수 area()의 리턴 값 출력
}
 parameter : 함수 내 지역변수
double area(int r) { // 함수 구현
 return 3.14*r*r; // 반지름 r의 원면적 리턴
}
```

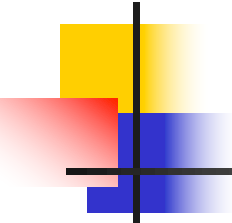
true는 1  
로 출력됨

#5.5-3hello1

n + 5 = 8

면적은 28.26





# namespace 개념

- 이름(identifier) 충돌이 발생하는 경우
  - 여러 명이 서로 나누어 프로젝트를 개발하는 경우
  - 오픈 소스 혹은 다른 사람이 작성한 소스나 목적 파일을 가져와서 컴파일 하거나 링크하는 경우
- 해결하는데 많은 시간과 노력이 필요
- namespace 키워드
  - 이름 충돌 해결
    - 2003년 새로운 ANSI C++ 표준에서 도입
  - 개발자가 자신만의 이름 공간을 생성할 수 있도록 함
    - 이름 공간 안에 선언된 이름은 다른 이름공간과 별도 구분
- 이름 공간 생성 및 사용

```
namespace alpha { // alpha 라는 이름 공간 생성
 // 이 곳에 선언된 모든 이름은 alpha 이름 공간에 생성된
 이름
}
```

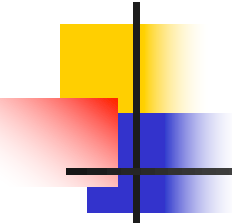
# std:: 란?

- std
  - ANSI C++ 표준에서 정의한 이름 공간(namespace) 중 하나
  - 모든 C++ 표준 라이브러리는 std 이름공간에 만들어져 있음
    - <iostream> 헤더 파일에 선언된 모든 이름: std 이름 공간 안에 있음
    - cout, cin, endl 등
  - std 이름 공간에 선언된 이름을 접근하기 위해 std:: 접두어 사용
    - std::cout, std::cin, std::endl
- std:: 생략
  - using 지시어 사용

```
using namespace std; // std 이름 공간에 선언된 모든 이름에 std:: 생략
.....
cout << "Hello" << endl; // std:: 생략
```

std:: 생략

std:: 생략



# #include <iostream>과 std

---

- <iostream>이 통째로 std 이름 공간 내에 선언
  - <iostream> 헤더 파일을 사용하려면 다음 코드 필요

```
#include <iostream>

using namespace std;
```

# C++ 프로그램에서 키 입력 받기

```
#include <iostream>

using namespace std;

int main() {
 cout << "너비를 입력하세요: ";
 int width;
 cin >> width; // 키보드로부터 너비를 읽어 width 변수에 저장
 cout << "높이를 입력하세요: ";
 int height;
 cin >> height; // 키보드로부터 높이를 읽어 height 변수에 저장
 int area = width*height; // 사각형의 면적 계산
 cout << "면적은 " << area << "\n"; // 면적을 출력하고 다음 줄로 넘어감
}
```

너비를 입력하세요: 3

높이를 입력하세요: 5

면적은 15

# cin과 >> 연산자를 이용한 키 입력

- cin
  - 표준 입력 장치인 키보드를 연결하는 C++ 입력 스트림 객체
- >> 연산자
  - 스트림 추출 연산자(stream extraction operator)
    - C++ 산술 시프트 연산자(>>)가 스트림 추출 연산자로 재정의됨
    - ostream 클래스에 구현됨 **DataType : istream**
    - 입력 스트림에서 값을 읽어 변수에 저장
  - 연속된 >> 연산자를 사용하여 여러 값 입력 가능

```
cout << "너비와 높이를 입력하세요: ";
cin >> width >> height;
cout << width << endl << height << endl;
```

>> 이 아니라 : 으로 수정

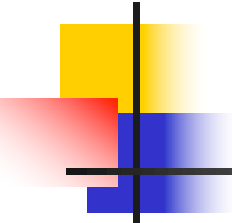
너비와 높이를 입력하세요>>23 36

23

36

width에  
입력

height에  
입력

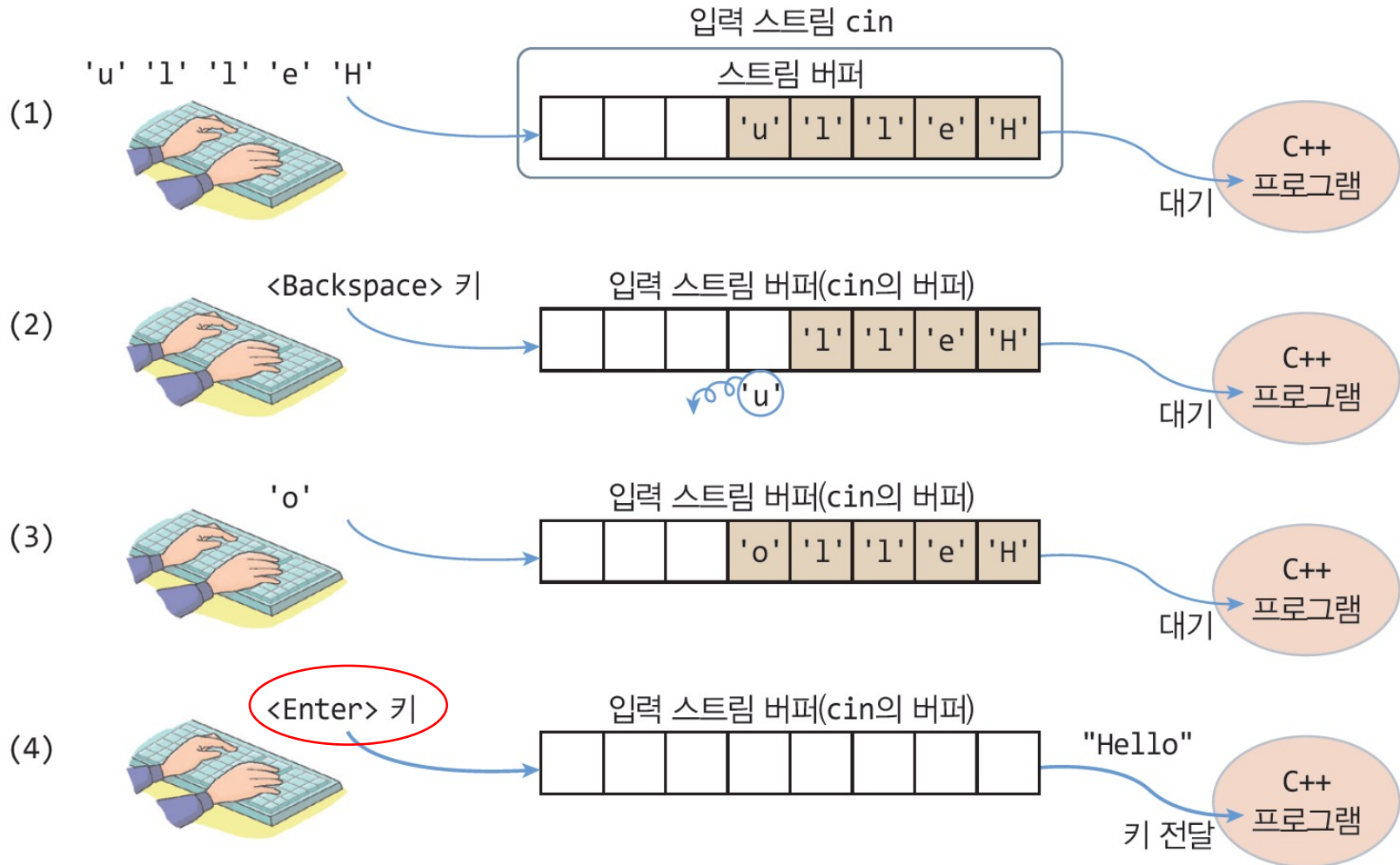


# <Enter> 키를 칠 때 변수에 값 전달

---

- cin의 특징
  - 입력 버퍼를 내장하고 있음 버퍼: 임시기억장치 엔터 누르는 순간 변수로 저장
  - <Enter>키가 입력될 때까지 입력된 키를 입력 **버퍼**에 저장
    - 도중에 <Backspace> 키를 입력하면 입력된 키 삭제
- >> 연산자
  - <Enter>키가 입력되면 바로소 cin의 입력 버퍼에서 키 값을 읽어 변수에 전달

# cin으로부터 키 입력 받는 과정



# 실행문 중간에 변수 선언

- C++의 변수 선언
  - 변수 선언은 아무 곳이나 가능

실행문 중간  
에 변수 선언

```
int width;

cin >> width; // 키보드로부터 너비를 읽는다.
cout << "높이를 입력하세요: ";

int height;

cin >> height; // 키보드로부터 높이를 읽는다.
// 너비와 높이로 구성되는 사각형의 면적을 계산한다.

int area = width*height;

cout << "면적은 " << area << endl; // 면적을 출력하고 한 줄 뗀다.
```



# C++ 문자열

- C++의 문자열 표현 방식 : 2가지
  - C-스tring 방식 - 'W0'로 끝나는 문자 배열

```
char name1[6] = {'G', 'r', 'a', 'c', 'e', '\0'}; // name1은 문자열 "Grace"
char name2[5] = {'G', 'r', 'a', 'c', 'e'}; // name2는 문자열이 아니고 단순 문자 배열
```

```
char name5[10] = "Grace";
```

| [0] | [1] | [2] | [3] | [4] | [5]  | [6]  | [7]  | [8]  | [9]  |
|-----|-----|-----|-----|-----|------|------|------|------|------|
| 'G' | 'r' | 'a' | 'c' | 'e' | 'W0' | 'W0' | 'W0' | 'W0' | 'W0' |

"Grace" 문자열

'\0' 로 초기화

문자 배열 마지막에 'NULL' 문자 넣을시 그것은 문자열이 된다

# cin을 이용한 문자열 입력

- 문자열 입력

```
char name[6]; // 5 개의 문자를 저장할 수 있는 char 배열
cin >> name; // 키보드로부터 문자열을 읽어 name 배열에 저장한다.
```

Grace

키 입력

name [0] [1] [2] [3] [4] [5]

|     |     |     |     |     |      |
|-----|-----|-----|-----|-----|------|
| 'G' | 'r' | 'a' | 'c' | 'e' | '\0' |
|-----|-----|-----|-----|-----|------|

“Grace” 문자열

# 키보드에서 문자열 입력 받고 출력

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
 cout << "이름을 입력하세요: ";
```

**\*\* space 가 seperate 역할을 함  
따라서 공백 전 첫번째 데이터만 기억**

```
 char name[11]; // 한글은 5개 글자, 영문은 10까지 저장할 수 있다.
```

```
 cin >> name; // 키보드로부터 문자열을 읽는다.
```

```
 cout << "이름은 " << name << "입니다\n"; // 이름을 출력한다.
```

```
}
```

이름을 입력하세요: Alpha

빈 칸 없이 키 입력해야 함

이름은 Alpha입니다

이름을 입력하세요: Al Pha

이름은 Al입니다

빈 칸을 만나면 문자  
열 입력 종료

# C-스트링을 이용하여 암호가 입력되면 프로그램을 종료하는 예

```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
 char password[11];
 cout << "프로그램을 종료하려면 암호를 입력하세요." << endl;
 while(true) {
 cout << "암호: ";
 cin >> password;
 if(strcmp(password, "C++") == 0) {
 cout << "프로그램을 정상 종료합니다." << endl;
 break;
 }
 else
 cout << "암호가 틀립니다~~" << endl;
 }
}
```

strcmp() 함수를  
사용하기 위한 헤더  
파일

프로그램을 종료하려면 암호를 입력하세요.

암호: Java

암호가 틀립니다~~

암호: C

암호가 틀립니다~~

암호: C++

프로그램을 정상 종료합니다.

빈 칸 없이 키 입력해야 함

# cin.getline()을 이용한 문자열 입력

- 공백이 낀 문자열을 입력 받는 방법
- cin.getline(<sup>parameter</sup>char buf[], int size, char delimiterChar)
  - buf에 최대 size-1개의 문자 입력. 끝에 '\0' 붙임
  - delimiterChar를 만나면 입력 중단. 끝에 '\0' 붙임
    - delimiterChar의 디폴트 값은 '\n'(<Enter>키)

```
char address[100];
cin.getline(address, 100, '\n');
```

최대 99개의 문자를 읽어 address 배열에 저장. 도중에 <Enter> 키를 만나면 입력 중단

default 값이 \n 이라서 안해줘도 됨!

사용자가 'Seoul Korea<Enter>'를 입력할 때,

address[0]   [1]   [2]   [3]   [4]   [5]   [6]   [7]   [8]   [9]   [10]   [11]   .....[99]

|     |     |     |     |     |     |     |     |     |     |     |      |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|-----|-----|
| 'S' | 'e' | 'o' | 'u' | 'l' | ' ' | 'K' | 'o' | 'r' | 'e' | 'a' | '\0' | ... | ... |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|-----|-----|

"Seoul Korea" 문자열

# cin.getline()을 이용한 문자열 입력

```
#include <iostream>
using namespace std;

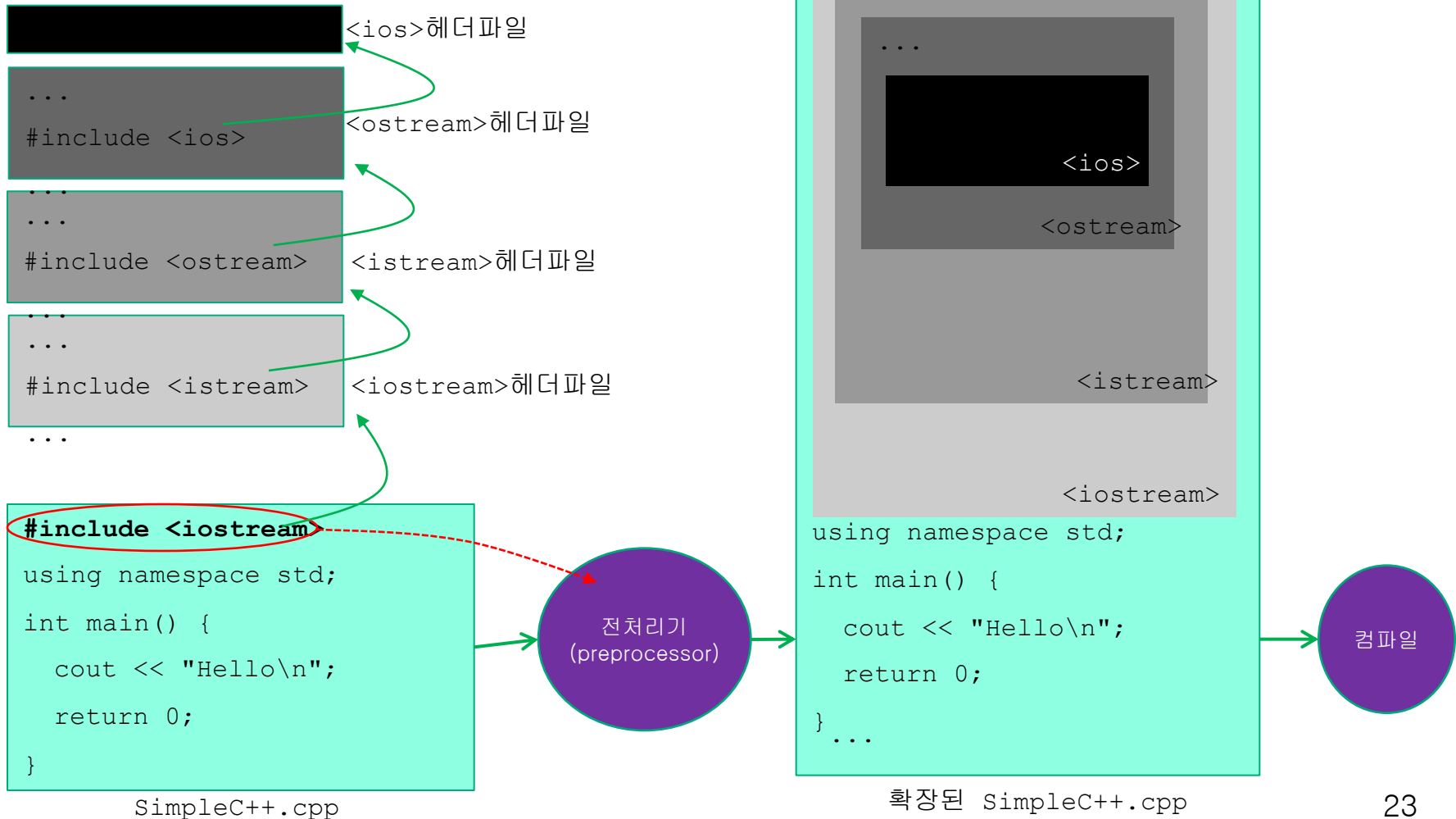
int main() {
 cout << "주소를 입력하세요: ";

 char address[100];
 cin.getline(address, 100, '\n'); // 키보드로부터 주소 읽기

 cout << "주소는 " << address << "입니다\n"; // 주소 출력
}
```

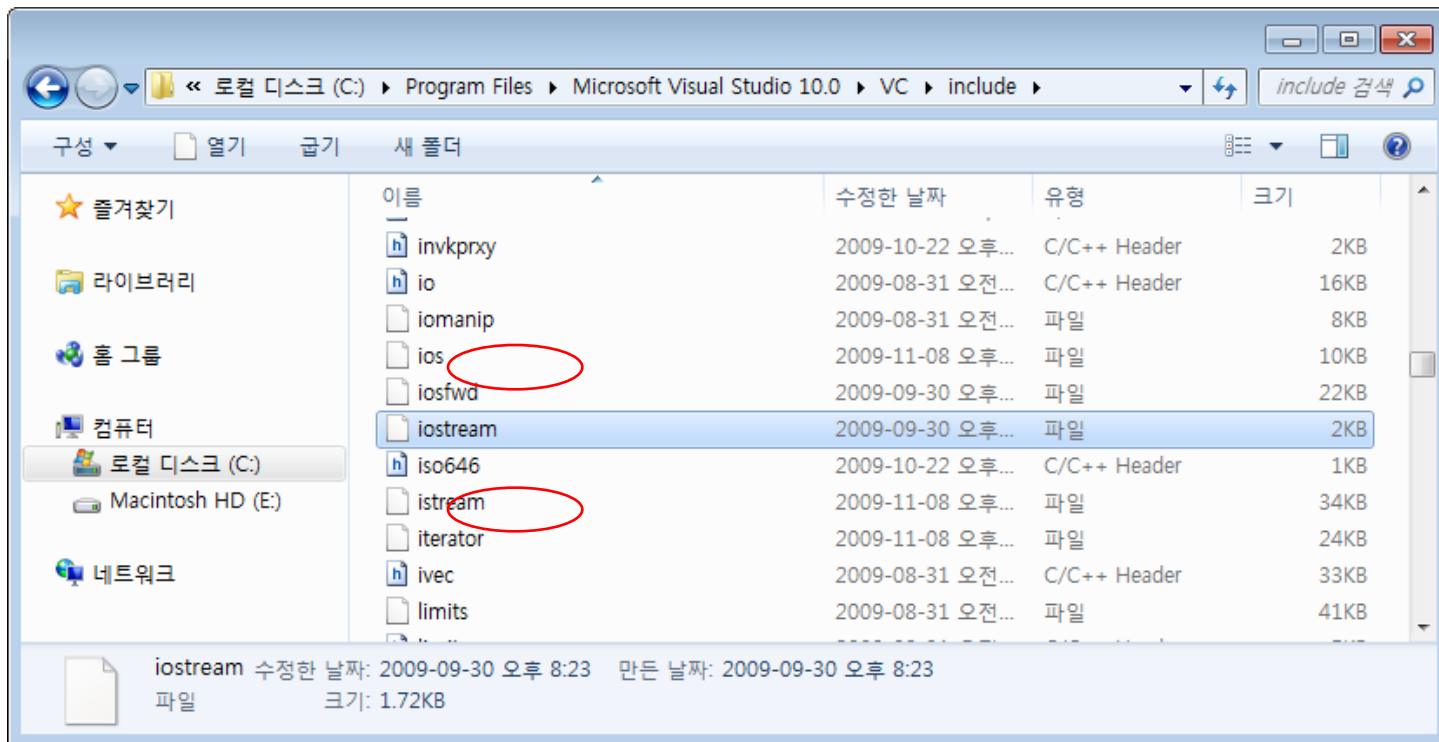
빈칸이 있어도 <Enter> 키가 입력될 때까지 하나의 문자열로 인식

# #include <iostream>와 전처리기



# <iostream> 헤더 파일은 어디에?

- <iostream> 파일은 텍스트 파일
- 컴파일러가 설치된 폴더 아래 include 폴더에 존재
  - C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.31.31103\include

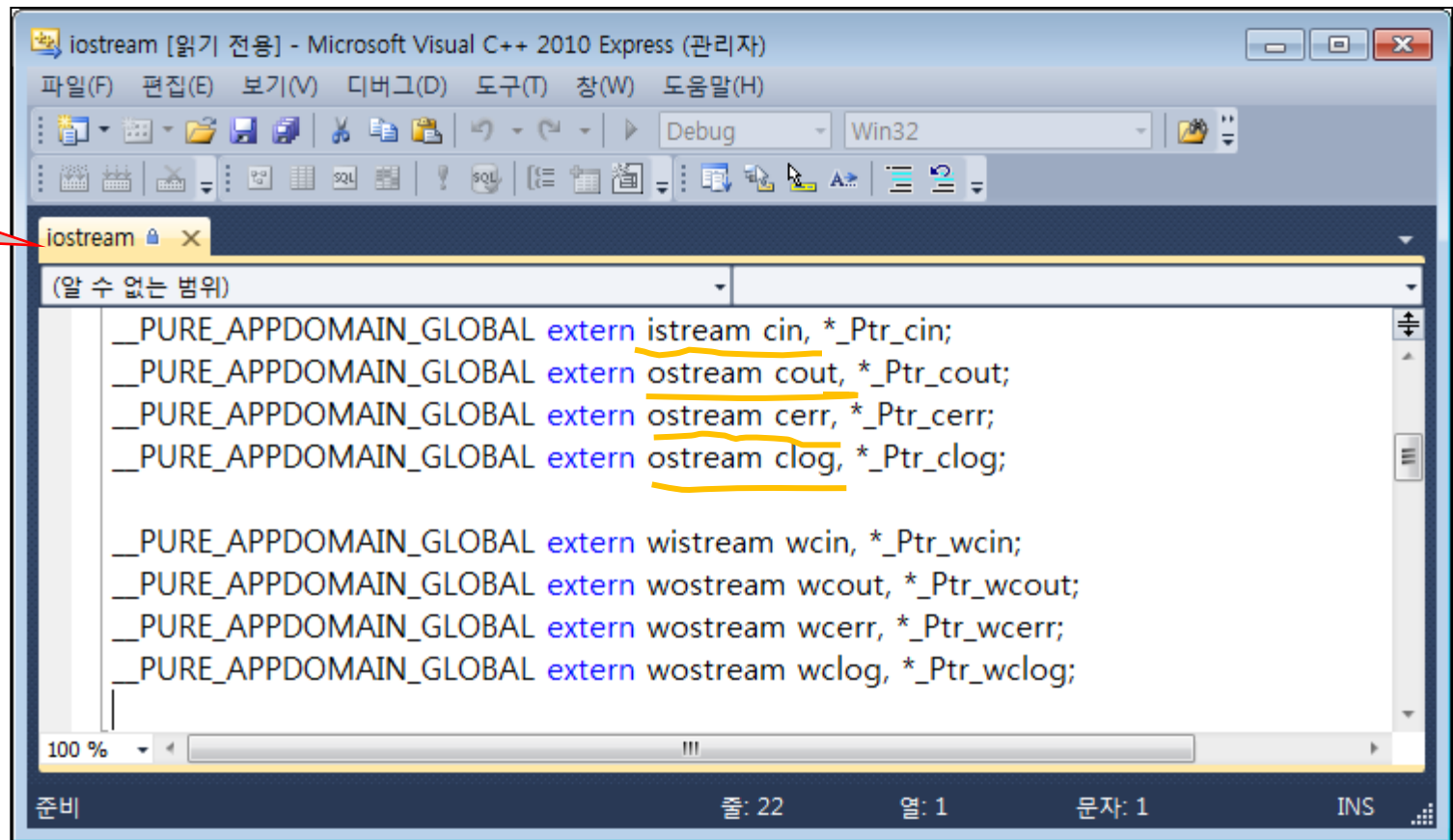




# cin과 cout은 어디에 선언되어 있는가?

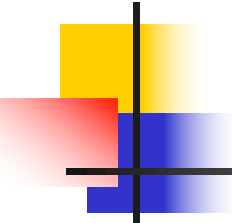
- cout이나 cin은 모두 <iostream>에 선언된 객체

<iostream> 헤더 파일



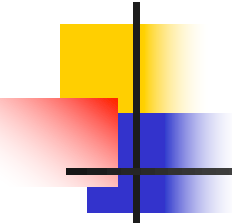
```
iostream [읽기 전용] - Microsoft Visual C++ 2010 Express (관리자)
파일(F) 편집(E) 보기(V) 디버그(D) 도구(T) 창(W) 도움말(H)
Debug Win32
iostream
(알 수 없는 범위)
_PURE_APPDOMAIN_GLOBAL extern istream cin, *_Ptr_cin;
_PURE_APPDOMAIN_GLOBAL extern ostream cout, *_Ptr_cout;
_PURE_APPDOMAIN_GLOBAL extern ostream cerr, *_Ptr_cerr;
_PURE_APPDOMAIN_GLOBAL extern ostream clog, *_Ptr_clog;

_PURE_APPDOMAIN_GLOBAL extern wistream wcin, *_Ptr_wcin;
_PURE_APPDOMAIN_GLOBAL extern wostream wcout, *_Ptr_wcout;
_PURE_APPDOMAIN_GLOBAL extern wostream wcerr, *_Ptr_wcerr;
_PURE_APPDOMAIN_GLOBAL extern wostream wclog, *_Ptr_wclog;
```



# #include <헤더파일>와 #include "헤더파일"

- #include <헤더파일> <시스템이 제공해주는 헤더파일>
  - ‘헤더파일’을 찾는 위치
    - 컴파일러가 설치된 폴더에서 찾으라는 지시
  - #include <iostream>
    - <iostream> 헤더 파일은 컴파일러가 제공
- #include “헤더파일” “직접만든 헤더파일”
  - ‘헤더파일’을 찾는 위치
    - 개발자의 프로젝트 폴더
    - 개발자가 컴파일 옵션으로 지정한 include 폴더에서 찾도록 지시



## tips

---

- Input buffer를 flush할 때의 명령어
  - Visual Studio old version
    - `fflush(stdin);`
  - Visual Studio 2015 이상
    - `cin.clear();`
    - `cin.ignore();`